THINKST SCAPES

Q3 • 2023

https://thinkst.com/ts

Contents

Cover photo: *Taktsang Palphug (Tiger's Nest) monastery clings to cliffs above the forested Paro Valley of Bhutan. Image by Dev Dua.*

Introduction

Welcome to the Q3, 2023 edition of ThinkstScapes! *This issue focuses on content released, published or presented between the first of July and the end of September, 2023.*

Quarter 3 showed a strong contingent of both content presented at the "Hacker summer camp" and some strong blog content. We suspect that in this quarter, talks not selected by the top conferences were released via blog, and that we'll see some of this content later in the year at regional conferences. Regardless of venue, there's some great content this quarter!

As a reminder: if you are aware of work we've missed, a blog post we should have seen or a conference we should have covered, we'd love to hear about it. Please send them to **ts@thinkst.com**

Each selected piece includes links to available papers, presentations or code repos.

In addition to over 1,000 blog posts, this quarter's content was drawn from the following conference presentations:

Conference name	Number of talks
Black Hat USA	109
Blue Team Con	30
USENIX Security	367
Texas Cyber Summit	124
Hack In Paris	19
28th European Symposium on Research in Computer Security	96
Pass the SALT	25
HITBSecConf – Phuket	27
44CON	44
SEC-T	19
NULLCON – GOA	25
BSidesLV	121
DEF CON	107
GrrCON	61
Total	**1174**

The majestic Sneffels range from Ridgway, CO. Image by Jacob Torrey.

Themes covered in this issue

CRYPTOGRAPHY STILL ISN'T EASY

This theme covers works that expose implementation flaws in what should be well-understood cryptography protocols, and highlights novel ways in which cryptography can improve security. Look for issues with mTLS and normal TLS certificate handling issues, cryptographic challenges to prevent phishing, a user-friendly way to verify a system's boot integrity, and even a compiler for writing applications that operate on fully-encrypted data.

[CONTINUED] ATTACK OF THE SIDE-CHANNELS

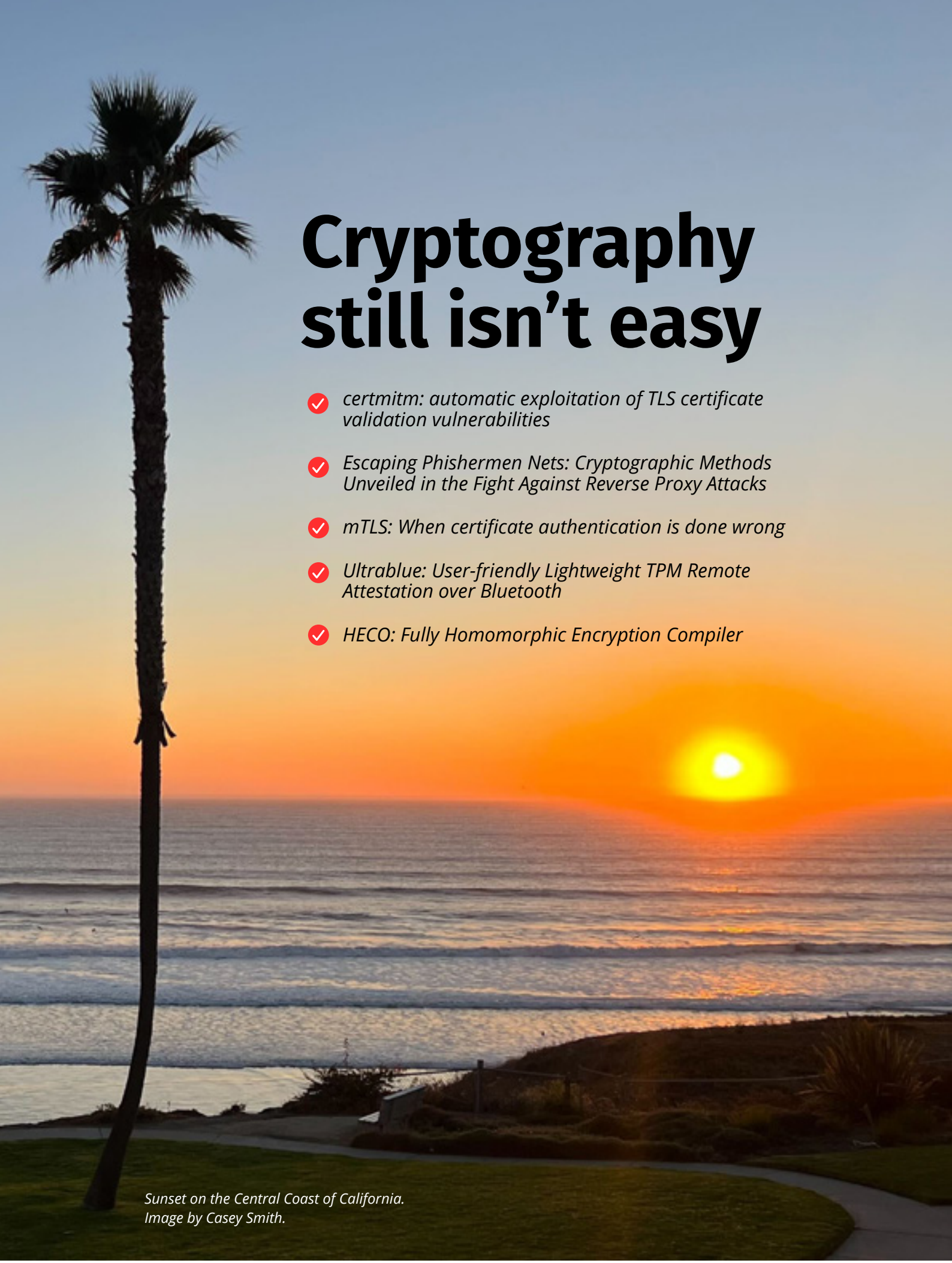
Works in this theme showcase the diversity of side-channel attacks, and how machine learning is boosting the signal-to-noise ratio, increasing the impact of these types of attacks. We include timing side-channels on SMS for locating a cell phone, scanning a local network from the browser, and another microarchitectural weakness with big consequences.

COMPOSITION IS HARD IN THE CLOUD

As our world is ever more connected and operated by third party infrastructure providers, getting the composition of services and configurations right is a challenge. This theme covers weaknesses in Cloudflare, GitHub and Microsoft Azure.

NIFTY SUNDRIES

As always, there was innovative research this quarter that didn't fit into any of the aforementioned themes, but warranted inclusion. Look for RCE over NFC, weaknesses in EDR updates, bugs in BGP, a large-scale war-driving for Bluetooth, and using LLMs to write fuzzing harnesses automatically.



Cryptography still isn't easy

- *certmitm: automatic exploitation of TLS certificate validation vulnerabilities*
- *Escaping Phishermen Nets: Cryptographic Methods Unveiled in the Fight Against Reverse Proxy Attacks*
- *mTLS: When certificate authentication is done wrong*
- *Ultrablue: User-friendly Lightweight TPM Remote Attestation over Bluetooth*
- *HECO: Fully Homomorphic Encryption Compiler*

Sunset on the Central Coast of California.
Image by Casey Smith.

certmitm: automatic exploitation of TLS certificate validation vulnerabilities

***Author**: Aapo Oksman*

Slides

Code

Video

This work built automation to identify certificate validation issues in libraries or applications that are not typically used by browsers. Even with robust certificate processing libraries, many defaults may be set incorrectly, or the library could be used in a way that results in invalid certificates being accepted – opening the door for man-in-the-middle attacks. The author discovered a couple of years ago that a bug in a Microsoft library would always return true when asked to inspect a certificate, allowing for self-signed or incorrectly signed certificates to pass muster. To ease the search for other instances of these types of validation bugs, they wrote certmitm.

Certmitm allows for rapid interception of TLS traffic, and tries passing a variety of malformed and incorrect certificates to the application under test to ensure that none of the self-generated certificates are allowed. In the initial testing and usage of certmitm, multiple bugs were identified, even if a majority of them did not result in CVEs/bug bounties due to the scoping of the bounty programs.

TAKEAWAYS:

- **While the consolidation of browsers** has led to more well-tested and safe certificate handling libraries, this research shows that many non-browser certificate libraries still have non-safe defaults. Using this tool in your development workflow should help ensure that insecure defaults are overridden, and that there are no surprises for application users.
- **Of the CVEs issued when using certmitm**, almost 60% were in Microsoft or Apple products, two vendors that shouldn't have these types of issues. It is concerning that the main commercial operating system (and associated ecosystems) vendors still have development teams publishing insecure-by-default applications and libraries.

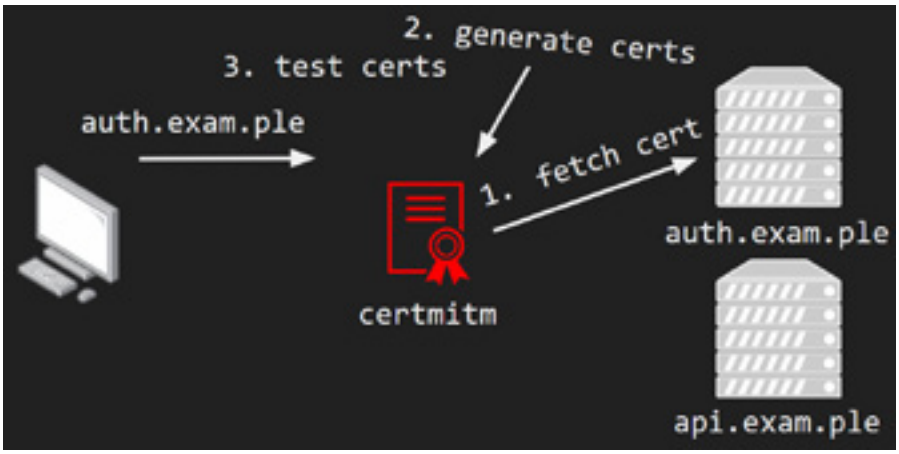


***Figure 1.** A figure showing how certmitm sits in the middle of the network connection between an application under test and servers to generate a variety of flawed certificates to check for improper validation.*

Escaping Phishermen Nets: Cryptographic Methods Unveiled in the Fight Against Reverse Proxy Attacks

Author:
Ksandros Apostoli

Blog

This blog post explored modern phishing techniques, known as Adversary-in-the-Middle (AitM), where instead of a static clone of the phished site, there is a dynamic proxy that rewrites requests and handles the TLS seamlessly. While there are some simple defences available (e.g. Thinkst's Cloned website token), more sophisticated AitM setups can rewrite Javascript to bypass the checks to see if the document was loaded from an unexpected domain.

By using the password hash (known to the server and the phished user as a shared secret, the server can encrypt expected domains or challenges in such a way that it cannot be rewritten unless the adversary has the password (in which case, phishing is not very valuable). Then the client can use the password hash to decrypt the challenge and verify the domain is as expected. The author notes that, although the goal of this approach is to raise the attacker's costs, it is still possible to modify the Javascript sent to the client to instead send the password directly to the adversary, but that requires more specific modifications and could alert the legitimate server to then suspend that user until the credentials can be refreshed.

TAKEAWAYS:

- **While these strategies will raise the cost for attackers,** FIDO2 hardware-based keys (including integrated Passkeys) prevent against both cloned site and AitM phishing. It is probably best to invest in improving to a cryptographically secure, standards-based authentication scheme than implementing a one-off defence, as both will require changes to the application.
- **These types of defences can offer insight** into phishing attempts targeting your organisation. Our free Cloned website token quickly lets you implement and deploy a simple script to your web applications to get early alerts about phishing campaigns; even possibly alerting you when the attacker is testing their infrastructure prior to sending out the first phishing link.

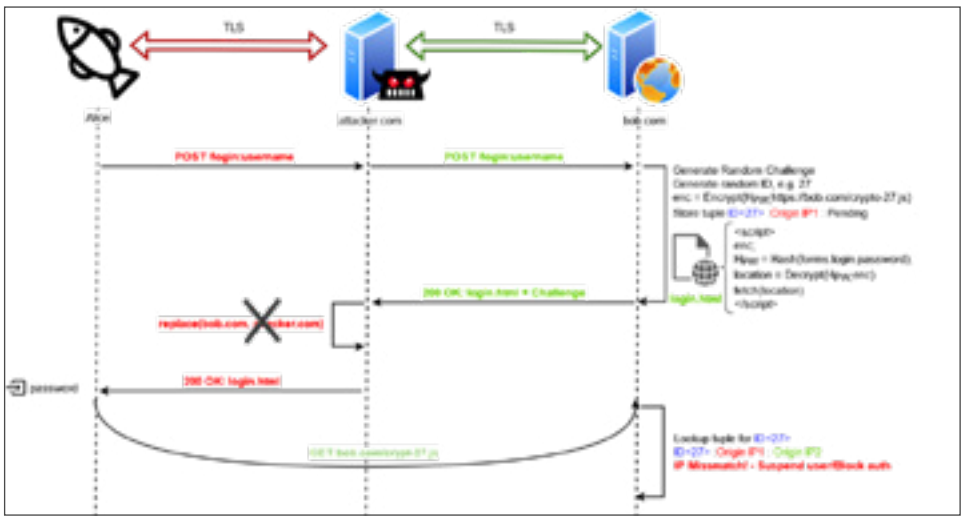


Figure 2. *A figure showing the proposed request and response flow that detects an AitM phish.*

mTLS: When certificate authentication is done wrong

Author:
Michael Stepankin

Slides

Blog

This research explores weaknesses in modern mTLS stacks. Despite excluding memory corruption and weak cryptography algorithms, vulnerabilities were discovered in how certificates were used, especially with trusting certificate properties prior to verification of signatures. When using a popular Java cryptography framework to validate certificates, all certificates provided to the library will be validated even if only the first certificate chain is properly signed. This allows for an attacker to inject invalid certificates later in the certificate bundle that will be treated as valid by the application. Other issues discovered involved injecting LDAP into certificate properties that could allow for LDAP injection, or even sending LDAP credentials to an attacker-controlled endpoint.

TAKEAWAY:

- **It is a promising sign that researchers** have to move beyond self-built cryptography and purposefully-weakened export algorithms to find vulnerabilities. However, it's concerning that libraries come with so many options that can introduce vulnerabilities depending on usage. As more orchestration frameworks implement their own CAs and mTLS, there are more opportunities for edge cases in certificate handling to have unintended side-effects.

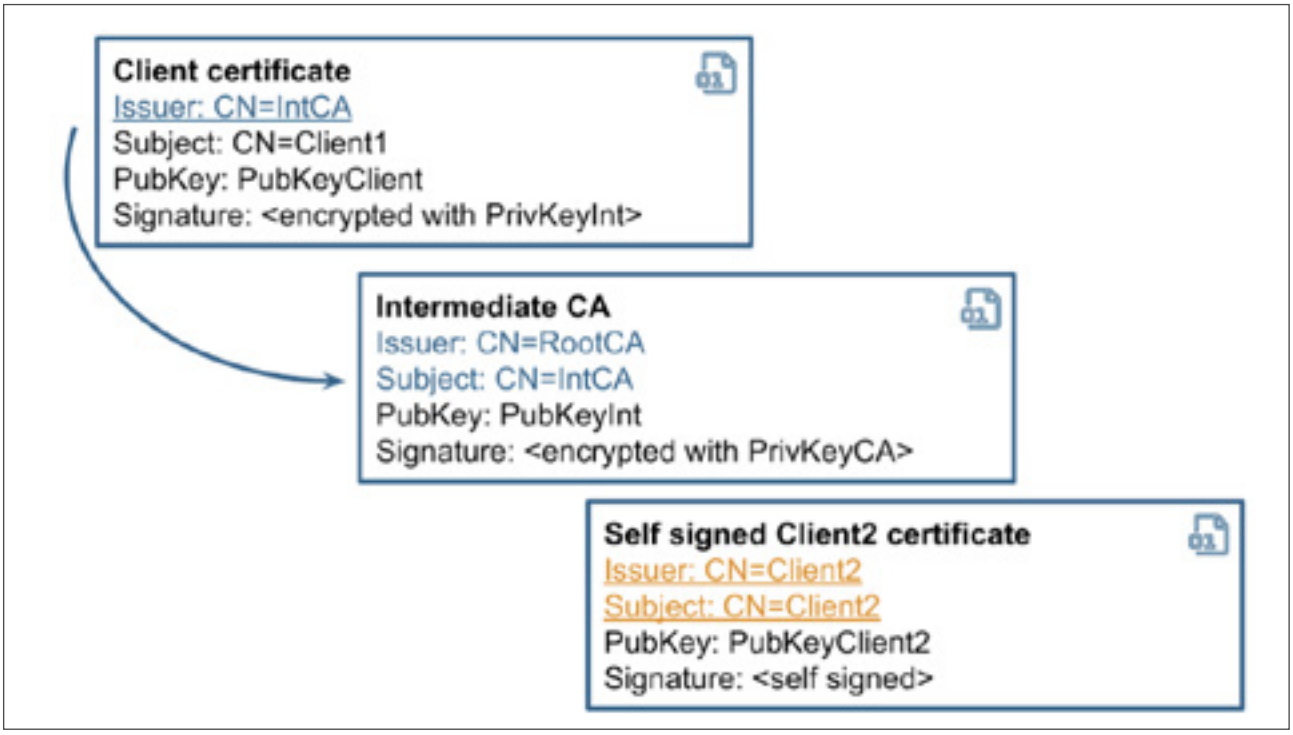


Figure 3. *A figure showing a combination of a valid certificate chain with a self-signed certificate for another client, an edge case that in some Java implementations will allow you to log in as the other client.*

Ultrablue: User-friendly Lightweight TPM Remote Attestation over Bluetooth

Authors: *Nicolas Bouchinet, Loïc Buckwell, and Gabriel Kerneis*

Slides

Code

Video

Trusted platform modules (TPMs) have been embedded into laptops and desktops for years, but have not been adopted en masse. Ultrablue looked at the UX hurdles to increase adoption, and built a solution to perform remote attestation of a device's state to a mobile device over Bluetooth. If the attestation process passes, a credential for an encrypted volume is sent to the device. Without a remote server verifying the boot environment, it is difficult to contextualise any deviations and to manage the regular update process. Same-device trusted computing can be fragile, as a single bit-flip in any measured code block results in an unbootable device.

Ultrablue ties a device and its state to a mobile phone app that stores the expected state as well as an encrypted credential for the device's encrypted volume(s). When the computer boots, it connects to the mobile phone over Bluetooth, performs a remote attestation challenge with the specified state variables stored in the TPM, and if the device is in the expected state, will allow it to boot. If there are deviations from the expected state, the app provides some context for those changes, and allows the user to allow it once, set it as the new baseline, or deny sending the credential. This results in the device falling back to a recovery password prompt to mount the encrypted volume(s).

TAKEAWAYS:

- **Trusted computing primitives** have been available for a long time, but are rarely used outside of vertically-integrated hardware platforms (e.g. Chromebooks), and almost never with remote attestation. Ultrablue opens the door for low-friction remote attestation against a growing number of firmware/pre-boot vulnerabilities, with the break-glass ability to type the password to unlock the encrypted volume(s) in the event that the mobile device is not available.
- **One of the challenges** with trusted computing at scale is handling updates and managing the availability/integrity risks. The ability to see what has changed, and with one click update the trusted baseline from a mobile device, should help with that.

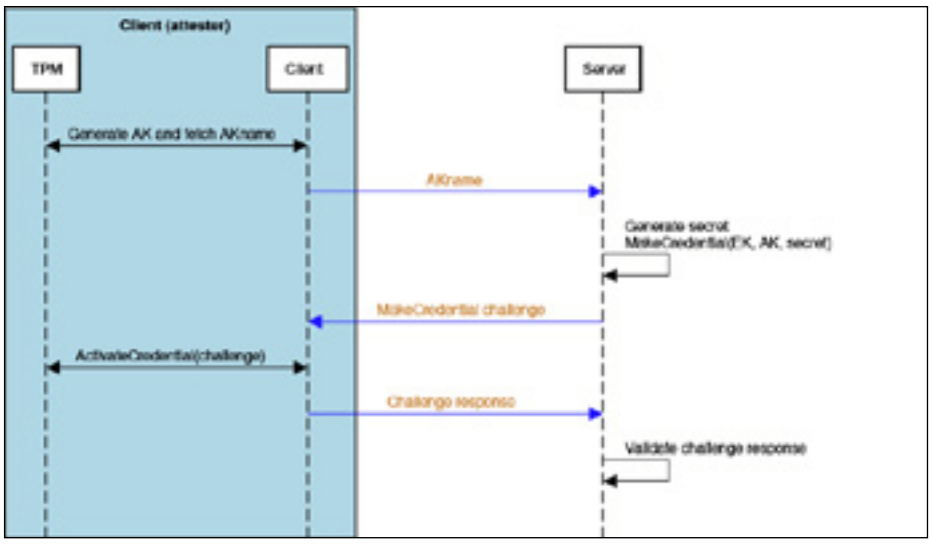


Figure 4. A state diagram of how the client enrols a credential with the server (running on a mobile phone).

HECO: Fully Homomorphic Encryption Compiler

Authors: *Alexander Viand, Patrick Jattke, Miro Haller, and Anwar Hithnawi*

Slides

Paper

Code

HECO aims to make development of applications that make use of fully-homomorphic encryption (FHE) easier. FHE allows for software to perform operations on encrypted data, but relies on cryptographic operations that can rapidly become non-performant if the algorithms using them have not been highly-optimised for FHE. There have been a few examples of using FHE in production, but each has required significant optimisation by experts in the FHE space.

HECO is a compiler that integrates into Python. It takes more traditional programs and successively optimises them automatically, reaching close to hand-optimised performance on example applications based on real-world use cases. With compilers like HECO and more optimisation passes that can translate from idiomatic imperative programs into FHE-optimised structures, more developers can write software that does not have access to sensitive customer or user data.

TAKEAWAYS:

- **While there was a large splash** with Gentry's initial work on FHE, very little of the progress in the field has made it into the mainstream consciousness. This work points to both examples of HE in consumer software (Edge's password monitor), and a viable path towards on-ramping more developers to this highly-complex and niche programming environment.
- **Expect to slowly see more** uses of FHE in the wild as these tools become more user-friendly and commonplace.

```python
def server(x_enc, y_enc, public_context):
  p = FrontendProgram()
    with CodeContext(p):
      def euclidean_sq(x: Tensor[8, Secret[int]],
                       y: Tensor[8, Secret[int]])
                      -> Secret[int]:
        sum: Secret[int] = 0
        for i in range(8):
          d = x[i] - y[i]
          sum = sum + (d * d)
        return sum

  # compile FHE code
  f = p.compile(context=public_context)
  # run FHE code using SEAL
  r_enc = f(x_enc, y_enc)
  return r_enc
```

Figure 5. Example Python code to compute Euclidean distance on encrypted data.

[Continued] attack of the side-channels

- *Freaky Leaky SMS: Extracting User Locations by Analyzing SMS Timings*
- *Downfall: Exploiting Speculative Data Gathering*
- *Your Clocks Have Ears — Timing-Based Browser-Based Local Network Port Scanner*

Twin Lakes, a pair of glacier-carved alpine lakes about 15 miles south of historic Leadville, Colorado. Image by Casey Smith.

Freaky Leaky SMS: Extracting User Locations by Analyzing SMS Timings

***Authors**: Evangelos Bitsikas, Theodor Schnitzler, Christina Pöpper, and Aanjhan Ranganathan*

Paper

Code

This work explored training an ML model on the round-trip-times (RTTs) from sending an SMS to a target device and receiving a delivery report in order to use the model to predict the country and region of a target device. In the early GSM specifications (and carried forward into 4/5G specifications), SMS messages were used not only for sending short text messages, but also for communication between the network and each handset, therefore there are a number of features built into the cellular devices that can be used to obtain RTTs, namely:

- Silent SMS: SMS messages that have no content, and are not visible to the end-user; and
- Delivery receipts: An SMS message can request a delivery receipt that the recipient will automatically (and without user control) reply stating that the message has been received.

Past work has used silent SMSes to trigger a target device to communicate with the local cell towers, which in coordination with the carrier can be used to recover a fine-grained location. Freaky Leaky SMS used ML to learn the relationships between RTTs and location. In the first step of the Figure below, multiple devices were sent to multiple locations, and were sent silent SMSes with delivery receipts – the RTTs between each location pair and device type were stored in a database. This dataset was used to train a model that, when given a sender (attacker) location and RTTs from the victim device, could predict that device's location. The accuracy of the predictions varied from ~75%–95% depending on the density of locations sampled and the cellular carrier used – one interesting discovery was that there was not a direct correlation between RTTs and distance; other factors must be at play.

TAKEAWAYS:

- **We can take some amount** of relief in the coarse-grained locations obtained through this approach, however the fact that every mobile device will still respond to SMSes automatically upon receipt, and that this can be done without any user awareness, is concerning. This was exposed in 2009 in Miller and Mulliner's iPhone fuzzing work, and continues to be carried forward into the LTE and 5G specifications.
- **While most home** routers do not respond to ICMP Pings on their WAN port, the legacy tail of mobile devices continues, even into 5G specifications.

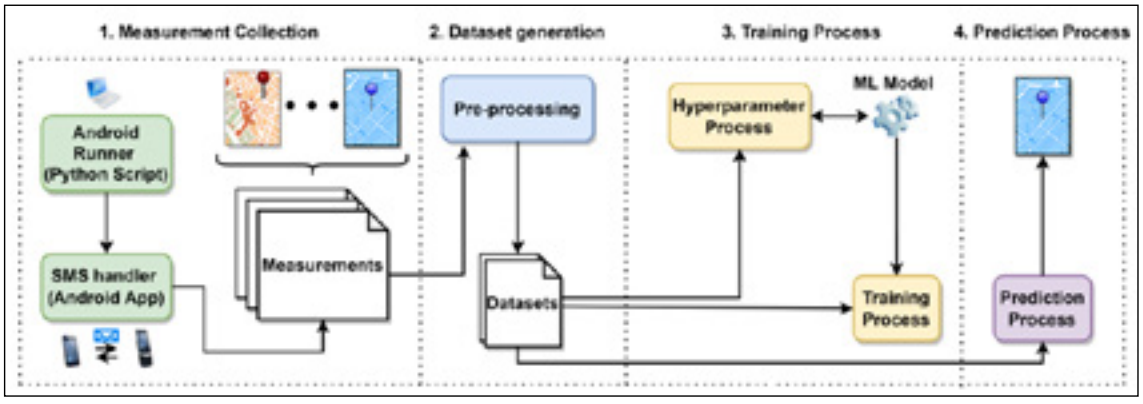


Figure 6. A diagram of the high-level process for training and prediction device locations.

Downfall: Exploiting Speculative Data Gathering

***Author**: Daniel Moghimi*

Code

Paper

This research uncovers another microarchitectural data leak issue with modern Intel CPUs, including the CPU revisions that fixed the previous bevy of vulnerabilities (e.g. Spectre, Meltdown, LVI, etc.). The researcher discovered that CPU cores share buffers for data being loaded into SIMD registers; this buffer can be influenced even by speculative execution. Building on a timing side-channel, the author was able to build a number of capabilities, including stealing AES keys from another VM, leaking data from the kernel to userspace, and violating Intel SGX's confidentiality guarantees.

TAKEAWAYS:

- **Despite the decline** in pace of microarchitectural attacks, in order to continue to compete on performance, more speculation and more shared caches will be discovered. This attack can be high-consequence, and others may be less so, but the whack-a-mole of microarchitectural issues will continue for a while. The continued challenges of wrangling these vulnerabilities are most pertinent to multi-tenant environments such as a cloud, browsers and container environments where attackers can control workloads.
- **While this attack** has not been demonstrated via Javascript in a browser, there does not appear to be any fundamental reason for the attack to not work via browser. This will increase the risk aperture, and further emphasises the need to restrict third party code execution, e.g. via ad blockers or browsing policies.

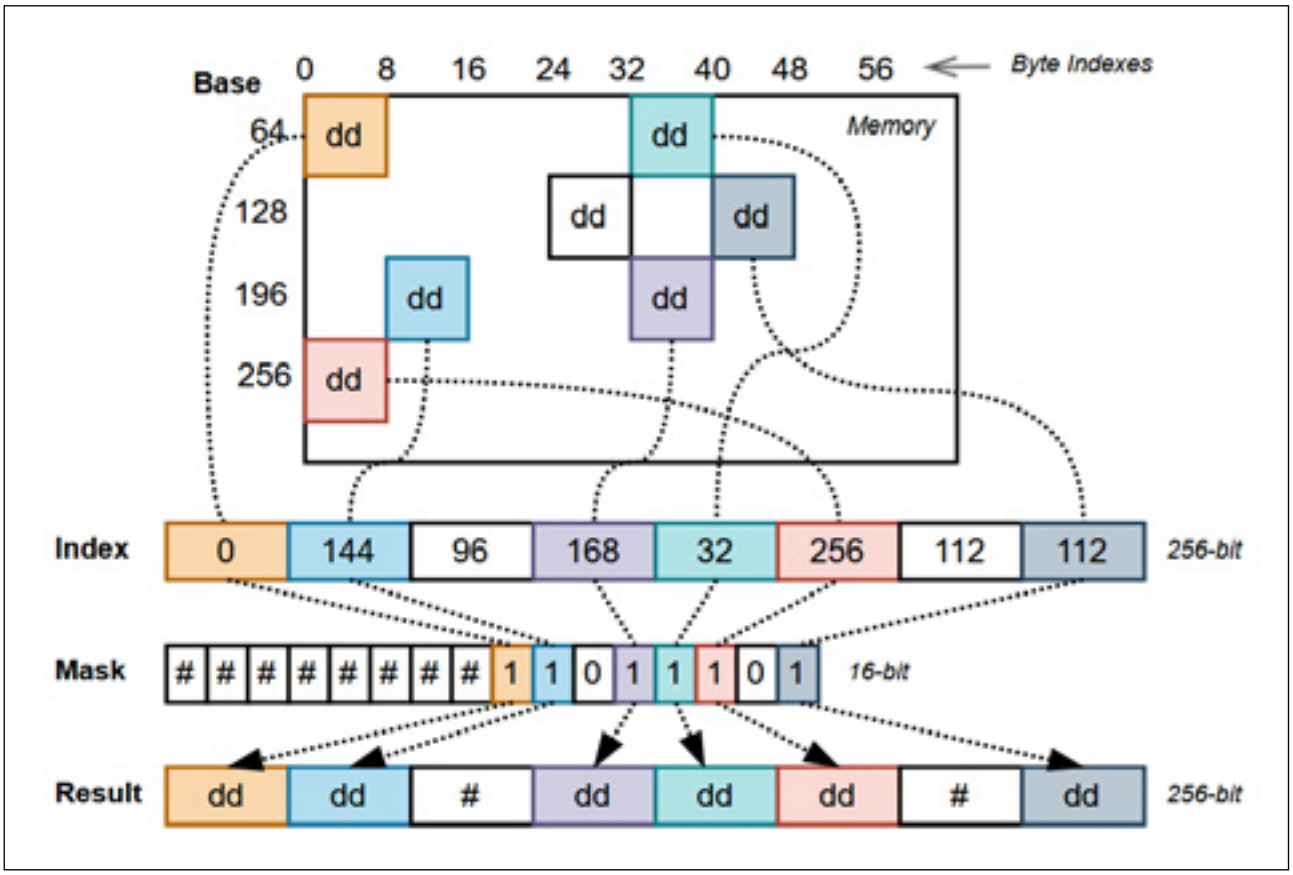


Figure 7. A figure showing how the Gather instruction works to collect non-contiguous data on Intel systems.

Your Clocks Have Ears – Timing-Based Browser-Based Local Network Port Scanner

***Author**: Dongsung Kim*

Slides

Video

Demo

In this work, a timing side-channel was exploited in order to build an in-browser port scanning capability to fingerprint or otherwise scan a victim browser. By surfacing the differences between a timeout, a successful connection and a CORS violation, the Javascript fetch function can be used with high-precision timing to determine which ports are open. This worked not only on the target system, but also for determining the LAN IP of the router and then scanning the local subnet. A full scan does take a little over an hour, but in a matter of a minute or so the local device can be scanned. The author notes that this works even if there is a VPN or other privacy protection in place, providing additional information to a malicious site owner about the identity of who may be browsing.

TAKEAWAYS:

- **While slow,** the ability to fingerprint a network behind a VPN or other privacy-preserving technique can be used to identify possible open ports to deanonymize targets. This is likely a very targeted scenario, but something to consider if there is a strong incentive to reveal an identity (e.g. black market participants).
- **That this pretty** simple timing side-channel is still able to reveal this information in browsers, speaks to the game of cat-and-mouse between privacy tools and deanonymization techniques.

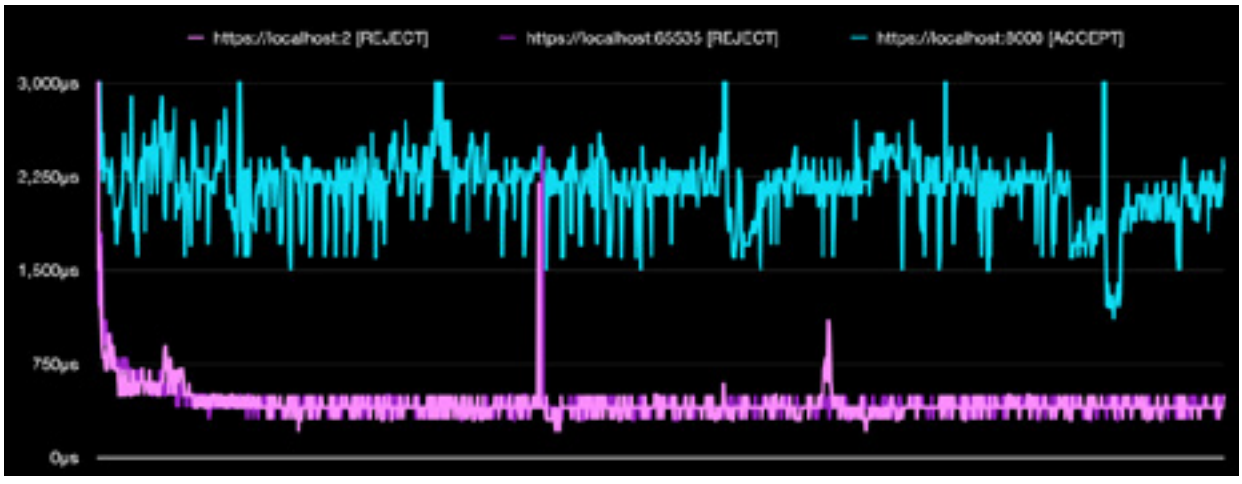


***Figure 8.** A chart showing the difference in timing between a fetch to a local port that is closed versus open.*

Composition is hard in the cloud

- *Using Cloudflare to bypass Cloudflare*
- *The GitHub Actions Worm: Compromising GitHub repositories through the Actions dependency tree*
- *All You Need is Guest*

Jacarande trees in the streets of Pretoria, South Africa. Image by Bradley Jayanath.

Using Cloudflare to bypass Cloudflare

***Authors**: Florian Schweitzer and Stefan Proksch*

Blog

This blog post shows weaknesses in Cloudflare configuration that allows for an attacker to bypass Cloudflare's protections and target a victim's origin server. While these weaknesses are configuration-dependent, the web UI for configuring a tenant doesn't offer the more secure, but more difficult to manage, configuration that would protect the victim.

When configuring a tenant, the easier option is to configure the origin server to only accept Cloudflare TLS certificates or Cloudflare IP ranges, preventing an attacker from bypassing the Cloudflare protections.

However, by setting an attacker's domain in the attacker's Cloudflare tenant to point to the victim origin server, the request will be signed by Cloudflare, thus being passed on and processed by the victim server. While it is possible to configure the origin to only accept specific certificates from a specific tenant, this configuration option is only available via an API. Initially these reports were marked as informative, but after public disclosure they were re-evaluated as high severity and the web UI should see improvements.

TAKEAWAYS:

- **Users** (even technical ones who are managing a Cloudflare tenant) will follow the defaults, or easiest path in a majority of cases. When that default opens customers up to vulnerability, it is important to realise that there is a shared responsibility, and to act promptly. Wherever there are large allow-lists that permit any infrastructure access that is owned by an infrastructure provider, it is important to realise that this threat is viable.
- **As more infrastructure** is outsourced to third-party vendors, more of these allow-lists will have unintended impacts.

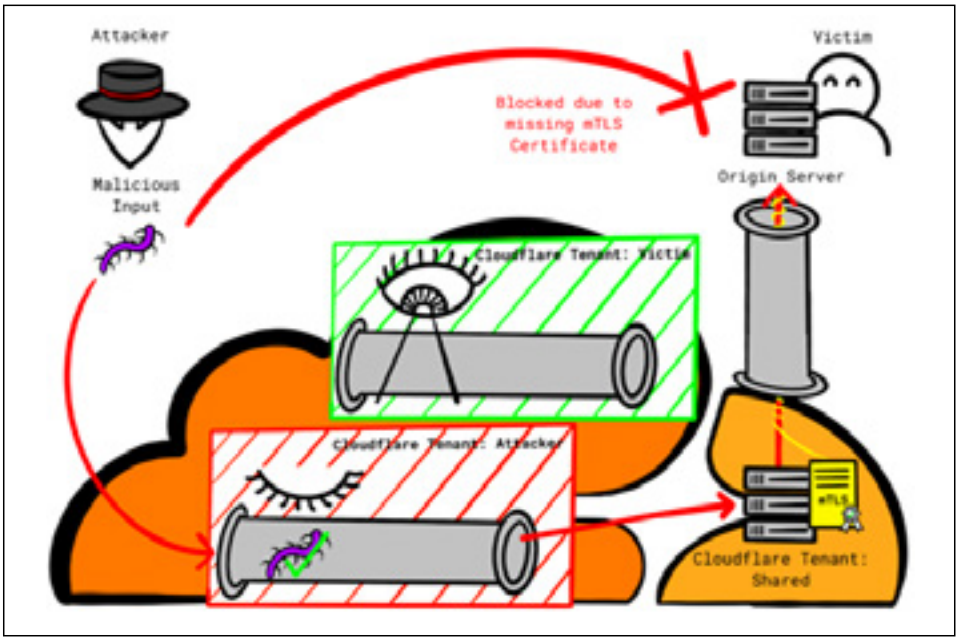


***Figure 9.** A diagram of how an attacker can route malicious inputs through their own tenant, which would be signed by a Cloudflare TLS certificate, generally trusted by other Cloudflare tenants.*

The GitHub Actions Worm: Compromising GitHub repositories through the Actions dependency tree

Author:
Asaf Greenholts

Slides

Blog

Video

This research explored the GitHub Actions CICD workflows, the marketplace of Actions, and the fact that Actions can include other Actions as a vector for large-scale supply chain attacks. The author built a graph database that showed the dependency trees of Actions repositories, and explored vulnerabilities that would allow for an initial access vector, such as a renamed GitHub user/organisation (repo-jacking), or an NPM package where the maintainer's email domain had expired. From that initial access (either by creating a GitHub account with the same name as the previous one that still had active references, or by registering the email domain and resetting passwords), a wormable structure was created.

Looking for repositories where the tokens in the Actions were configured to have read-and-write permissions, the Action can then overwrite its own code, so a vulnerable dependency can corrupt all Actions that use it. A single Action was discovered that allowed for repo-jacking, then that Action allowed for corruption of two other popular Actions that resulted in the potential access to thousands of repositories, and all the packages/releases from each.

Finally the researcher widened their search and found almost 200 repositories that were vulnerable to an initial access vector, transitively impacting thousands more repositories. A few defences have been put in place that make this type of attack more difficult (such as GitHub blocking others from reusing user/organisation names that have popular repositories), but some exposure can only be mitigated by changing the permissions in each Action or repository configuration.

TAKEAWAYS:

- **The scale of dependencies in modern** software is challenging enough; adding in the transitive dependencies of the Actions that build those packages further increases the scale of impact. A single example repository that was vulnerable to repo-jacking had impacts on thousands of repositories. The researcher identified almost 200 vulnerable repositories that each have their long tail of consuming actions.
- **The ability for a malicious action** to worm itself into actions that consume it is frightening; prompt restriction of GITHUB_TOKEN permissions is almost certainly a wise option when self-mutating Actions are not needed.

Figure 10. *A graph of impact from a single Actions repository that is vulnerable to repo-jacking and all the repositories that depend on it.*

All You Need is Guest

Author:
Michael Bargury

Slides

Code

This research explores Azure tenant guest access, a feature used by Azure customers to quickly invite non-members to join their tenant in a deny-by-default mode. Helpful for allowing them access to upload large files, or collaborate on specific projects, the author discovered a number of risks when coupled with Azure Power Apps, a low/no-code platform.

To simplify application creation, many applications are created with credentials of the user who built them, automatically masquerading as that user for access e.g. to datastores. By making these applications accessible to all users in the tenant (which seems reasonable since it's assumed that the tenant is populated with employees), guests now can access these applications.

From that initial discovery, the author found (and reported) further vulnerabilities allowing them to exfiltrate all the credentials from Power Apps, and dump the datastores they were tied to. This research was encapsulated in a tool, PowerPwn, which automates the scanning and enumeration of potentially-exposed applications and their credentials.

TAKEAWAYS:

- **Low-code/no-code platforms** increase the scale of applications being developed that handle sensitive data. AppSec and other security processes must account for these new developer populations who may not have been through the typical corporate onboarding for secure development.
- **The ability to quickly build** an application and share it across multiple Azure tenants, while also transparently including your credentials, will result in pain. Coupled with the seamless ability to add Azure users to your tenant as a guest, the risks are high of one person sharing an application with all users assuming they are only employees or NDA'd contractors, and another person adding an untrusted guest assuming there is nothing of value for them to access.

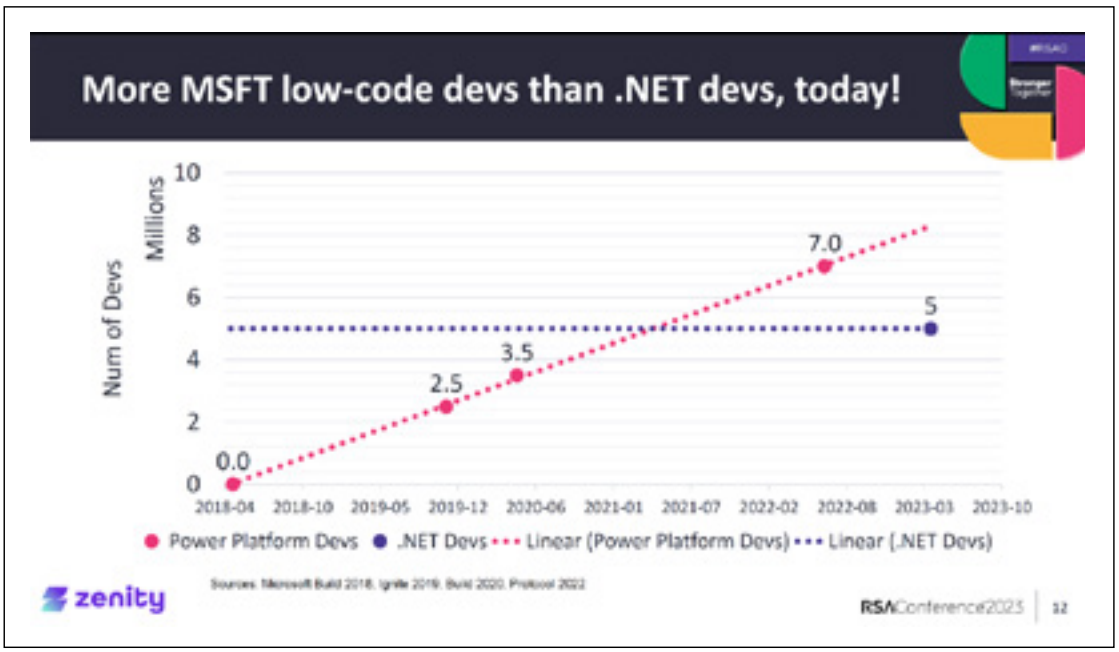


Figure 11. A chart showing the explosive growth of Azure Power Platform users/ developers compared with .NET developers.

Sandstone rock formations outside of Clarens in the Free State, South Africa. Image by Bradley Jayanath.

Nifty sundries

- *Contactless Overflow: Critical contactless vulnerabilities in NFC readers used in point of sales and ATMs*
- *Defender-Pretender: When Windows Defender Updates Become a Security Risk*
- *Fuzz target generation using LLMs*
- *Route to Bugs: Analyzing the Security of BGP Message Parsing*
- *It was harder to sniff Bluetooth through my mask during the pandemic...*

Contactless Overflow: Critical contactless vulnerabilities in NFC readers used in point of sales and ATMs

Author:
Josep Pi Rodriguez

Slides

Video

This research explored the NFC parsing firmware in a variety of NFC readers (typically integrated into point-of-sale systems), and discovered overflow vulnerabilities in a number of OEM products – resulting in full RCE. Starting with a firmware dump via JTAG, the author identified RCE vulnerabilities in almost 10 different OEMs, which are then integrated into thousands of devices, from payment terminals to vending machines.

In order to trigger the vulnerability, the author needed to be able to send a larger packet to the device than is typical (but is still within the NFC specifications). They discovered that Pixel Android devices allow more flexibility in their NFC code, allowing them to build an Android application that would be able to trigger the exploit.

A few demos were presented, including using a phone to trigger the vulnerability, then changing the logic on the device to change the payment amount for subsequent transactions, while reporting nothing out of the ordinary to neither the host system nor the LCD display. This presentation was finally released two years after initial disclosure to the vendors due to the sensitivity, impact, and difficulty to implement and deploy patches.

TAKEAWAYS:

- **Embedded devices** have a complex and opaque supply chain where components are relabeled and integrated into other devices. If critical vulnerabilities are discovered in these upstream OEM components/ firmware (which generally comes with limited to no update capabilities), these vulnerabilities will persist in the wild for many more years to come.
- **It is concerning** that a sensitive part of payment processing that acts as an aperture for untrusted RF would have buffer overflow vulnerabilities, but unfortunately, it is not surprising. These vulnerabilities and more like them will be ripe for exploitation for a long while.

Figure 12. *A figure showing a possible impact of getting code execution on an NFC payment terminal.*

Defender-Pretender: When Windows Defender Updates Become a Security Risk

***Authors**: Omer Attias and Tomer Bar*

Slides

Code

The signature update process is critical to software trust and integrity. In this talk, the researchers took a close look at the Windows Defender update process, as well as reverse engineering Defender's VDM files.

VDM files are Windows Portable Executable files that include a resource section with compressed data that comprises Defender signatures. The team discovered that Signatures in both Base and Delta VDM files are compressed but, surprisingly, not encrypted.

After decompressing the signatures in the Base file, they were able to easily identify where the signatures started and ended and were able to identify the actual malware string. During this process, they uncovered portions of the update that were not properly signed or encrypted. This allowed the team to forge entries. Furthermore, the researchers uncovered LUA scripts inside of the unsigned signature files that could allow for manipulation that leads to potential local privilege escalation, however, this was not fully demonstrated.

TAKEAWAYS:

- **Processes that run with high system privileges** need to endure tough scrutiny of the encryption and signing chain.
- **This is an excellent talk**, describing the research approach, dead ends and ultimate triumphs. The published research can be leveraged by others to build upon, which we believe is the raison d'être of publishing research.

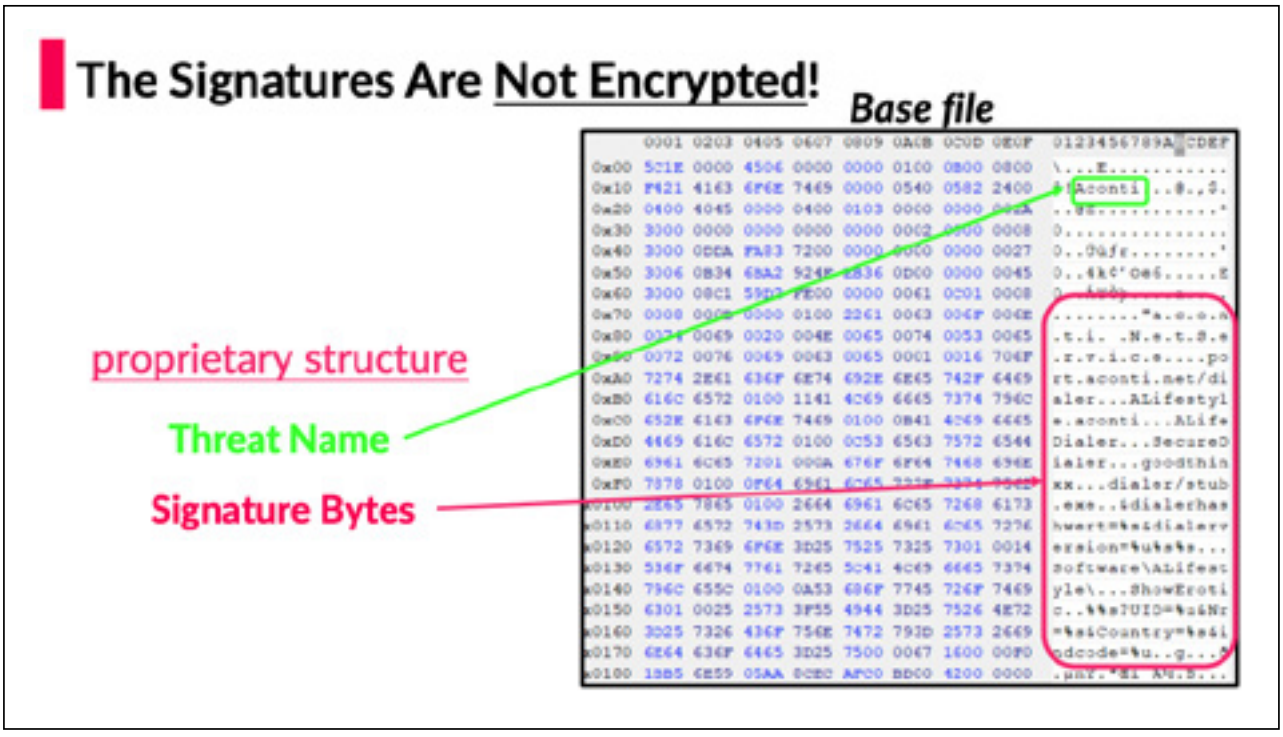


Figure 13. Showing the unencrypted portions of a Windows Defender Update package.

Fuzz target generation using LLMs

***Authors**: Dongge Liu, Jonathan Metzman, and Oliver Chang*

Results

Report

Blog

This work, by the Google open source security team (behind OSS-Fuzz), explored using generative AI/LLMs to automate the process of generating fuzzing harnesses for open-source projects.

The OSS-Fuzz project aims to continuously fuzz open-source projects to improve their security and reliability. Currently the most human-labour intensive part of the project is writing test cases or fuzzing harnesses to exercise diverse parts of the code under test – this research explored automating harness generation using LLMs.

By prioritising targets with low code coverage, a back-and-forth "dialogue" was created between the OSS-Fuzz framework and an LLM, iterating to get buildable harnesses that increased coverage. Over the majority of targets, the LLM was eventually able to get a harness synthesised that compiled, and depending on the target project, was able to increase coverage slightly (1-6%). Tinyxml2 was an outlier, seeing the most impressive results, an increase of coverage by over 30%.

TAKEAWAYS:

- **Given the high percentage of open-source code** in all applications, any way to automate more of the analysis is a positive. While the initial results mostly were marginal increases in code coverage, this is sure to improve over time.
- **By automating this** time-consuming and relatively low-skill part of the overall dynamic testing process, more human attention can be spent on the process of determining root-cause and exploitability of the discovered crashes. With time, more of these tasks will become within the realm of feasibility by AI/ML.

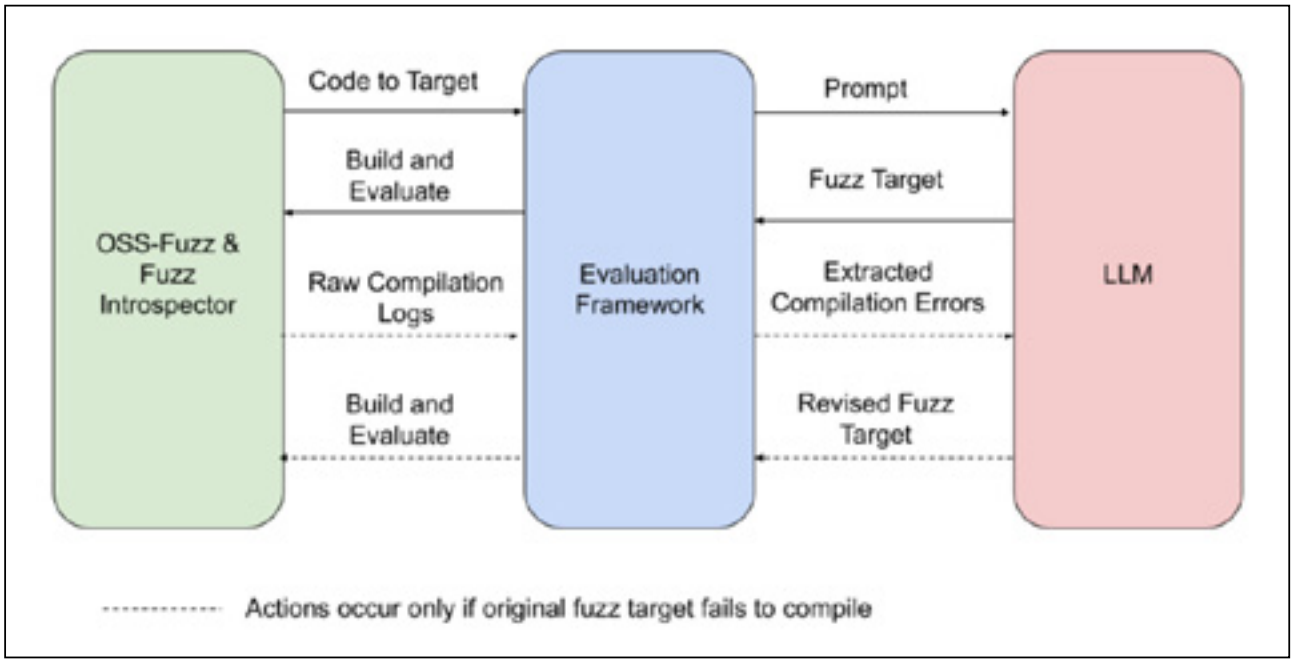


Figure 14. A figure showing how adding an LLM into the fuzzer test case generation process works to improve fuzzer coverage.

Route to Bugs: Analyzing the Security of BGP Message Parsing

***Authors**: Daniel dos Santos, Simon Guiot, Stanislav Dashevskyi, Amine Amri, and Oussama Kerro*

Slides

Code

This work explored the security of BGP routing services – not the known weaknesses inherent in the protocol itself, but application flaws related to parsing BGP messages. In their literature review, they only found a similar study from 20 years prior. Instead, the focus has been on how to best prevent rogue routing information from causing large-scale incidents. There have been a number of BGP parsing issues, but not a broad survey of the common software packages – until this research.

The authors built a prototype fuzzer specifically for BGP messages, and set it on one of the most popular BGP routing software applications, FRRouting (which has been forked and used in many other routing solutions). FRRouting is the routing component in both Microsoft's SONiC and the Linux Foundation's DENTOS, so it powers a large portion of the cloud enterprise. Quite quickly three distinct CVEs were reported, which allowed for a message to take down the router (without authentication needed). By repeating these messages, those routers would be kept out of operation, potentially causing large-scale networking impacts.

TAKEAWAYS:

- **It is concerning** how many years a critical protocol underpinning the internet has gone without the deeper analysis that a custom fuzzer can bring.
- **While BGP itself is simple enough,** there are other protocols and extensions that can increase the parsing complexity. Expect to see more vulnerabilities in these core services with additional scrutiny.

***Figure 15.** A table of the CVEs discovered in short order with the new fuzzer in a popular BGP implementation.*

CVE ID	Tested Product	Description	Potential Impact
CVE-2022-40302	FRRouting 8.4	Out-of-bounds read when processing a malformed BGP OPEN message with an Extended Optional Parameters Length option.	DoS
CVE-2022-40318	FRRouting 8.4	Out-of-bounds read when processing a malformed BGP OPEN message with an Extended Optional Parameters Length option. This is a different issue from CVE-2022-40302.	DoS
CVE-2022-43681	FRRouting 8.4	Out-of-bounds read when processing a malformed BGP OPEN message that abruptly ends with the option length octet (or the option length word, in case of OPEN with extended option lengths message).	DoS

It was harder to sniff Bluetooth through my mask during the pandemic...

***Author**: Xeno Kovah*

Slides

Data

This work explored the invisible world of Bluetooth (BT) and Bluetooth low-energy (BTLE) around us. While there was nothing theoretically new in this work, the empirical data collected about the reality of randomised/private BT addresses, the scale of BTLE deployments, and the workflow to interrogate a device, and then determine its internals (e.g. via FCC filings), show how omnipresent BT is in our environments. Combining the survey data the author collected while travelling with the open-source dataset WiGLE (which collects BT data as well as Wi-Fi) allowed for rapid hypothesis testing to find out more about those devices.

TAKEAWAYS:

- **The lower costs** and power consumption of BTLE have led to an incredible deployment of BTLE devices across the world. Coupled with firmware exploits and a very confusing supply chain where a small number of hardware OEMs are rebranded, the attack surface is enormous in scale, despite being lower risk due to requiring physical proximity. If your threat model includes (near) physical access, it would be worthwhile to explore your own BTLE footprint.
- **The scale of information** yet to be extracted from large-scale BTLE datasets like WiGLE is impressive. It means being able to quickly pivot from an address or naming scheme to a world-wide map showing the locations of those devices and how their pattern of life could offer significant business insights as well as aid in target selection for exploitation.

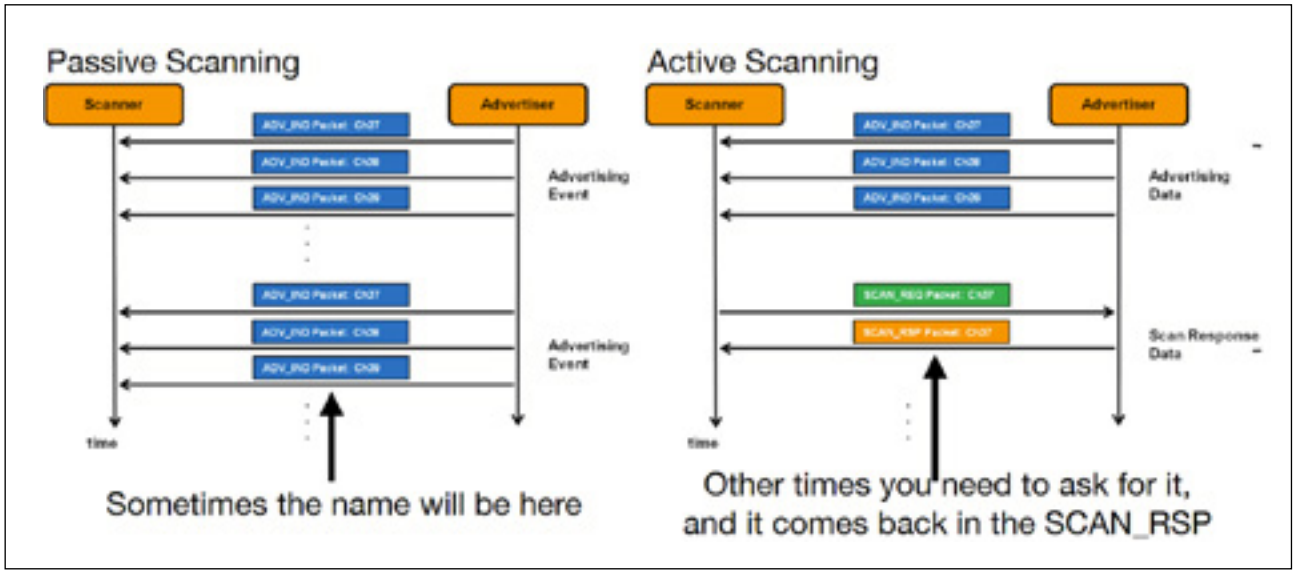


Figure 16. The two sources of data for device and manufacturer name to enrich Bluetooth address collections.

Conclusions

This was an impressive quarter. The research community has consistently put out great work in Q2 and Q3!

WE HIGHLIGHTED THREE UNIQUE THEMES THIS QUARTER:

1. Challenges and opportunities with cryptography.
2. ML-boosted side-channels.
3. Cloud-scale composition vulnerabilities.

Next quarter will conclude 2023! Look for a reflection on another year of ThinkstScapes, and how well we've done on drawing out the impactful research and top themes of the last 12 months.

Path lined with Aspens in Ridgway, CO, USA.
Image by Jacob Torrey.


THINKST
LABS